PYTHON ACTIVITY-BASED LEARNING

Sample Project Report

Submitted by

Sachin Kumar

Roll Number: 26001470

Branch: CSD **Section:** 2

Selected Case Studies: 10 Python Programs

This is an illustrative sample report. Review the assumptions, test the programs on your computer, and replace sample-output text with your own screenshots before submission.

Report Sections	Included in this sample
Problem statement and objectives	Yes
Assumptions, algorithm, pseudocode	Yes
Python source code	Yes
Test cases and sample output	Yes
Output screenshots	Insert your own screenshots

Contents

1. Progressive Electricity Billing

2. Attendance Recovery Computation

3. ATM Transaction Sequence Simulation

4. Fuel Consumption and Route Comparison

5. Advanced Numeric Pattern Generator

6. Password Policy Validation Engine

7. String-Based Examination Answer Evaluation

8. Text Compression Using Character Runs

9. Student Marks Matrix Analysis

10. Food Waste Monitoring System

Appendix: Submission checklist

Note: This report uses illustrative rules and sample data. Confirm that they match your assignment brief and instructor's expectations.

Case Study 1 — Progressive Electricity Billing

1. Problem Statement

Write a Python program that calculates an electricity bill from the number of units consumed, using progressive unit slabs and a fixed charge.

2. Objectives

• Read electricity units from the user.

• Calculate charges using conditional statements.

• Display energy charge, fixed charge, and total bill.

3. Assumptions

• Units must be a non-negative integer.

• Illustrative tariff: first 100 units at ■2/unit, next 100 at ■3/unit, remaining units at ■5/unit.

• Fixed charge is ■50.

4. Algorithm

1. Read units.

2. If units are negative, display an error.

3. Calculate slab-wise energy charge.

4. Add the fixed charge and display the bill.

5. Pseudocode

```
START
INPUT units
IF units < 0
    PRINT invalid input
ELSE
    CALCULATE slab-wise charge
    total = energy_charge + 50
    PRINT charges and total
END IF
STOP
```

6. Python Source Code

```
units = int(input("Enter electricity units consumed: "))

if units < 0:
    print("Invalid units")
else:
    if units <= 100:
        energy = units * 2
    elif units <= 200:
        energy = 100 * 2 + (units - 100) * 3
    else:
        energy = 100 * 2 + 100 * 3 + (units - 200) * 5

    fixed = 50
    print("Energy charge: Rs.", energy)
    print("Fixed charge: Rs.", fixed)
    print("Total bill: Rs.", energy + fixed)
```

7. Test Case

Input / Condition	Expected Result
250 units	Energy ■750; fixed ■50; total ■800
-5 units	Invalid units

8. Sample Output

```
Enter electricity units consumed: 250
Energy charge: Rs. 750
Fixed charge: Rs. 50
Total bill: Rs. 800
```

9. Conclusion

The program applies progressive slabs and adds the fixed charge to calculate the final electricity bill.

Case Study 2 — Attendance Recovery Computation

1. Problem Statement

Calculate the number of additional consecutive classes a student must attend to reach a target attendance percentage.

2. Objectives

• Accept classes attended and classes conducted.

• Calculate current attendance.

• Find the additional classes required to reach 75%.

3. Assumptions

• Target attendance is 75%.

• Every future class is attended.

• Total classes conducted must be greater than zero.

4. Algorithm

1. Read attended and conducted counts.

2. Validate the counts.

3. Increase the required-class counter until the target is reached.

4. Display the current percentage and required classes.

5. Pseudocode

```
START
INPUT attended, conducted
VALIDATE values
required = 0
WHILE (attended + required) / (conducted + required) < 0.75
    required = required + 1
PRINT current percentage and required
STOP
```

6. Python Source Code

```
attended = int(input("Classes attended: "))
conducted = int(input("Total classes conducted: "))
target = 75

if conducted <= 0 or attended < 0 or attended > conducted:
    print("Invalid attendance details")
else:
    required = 0
    while (attended + required) * 100 < target * (conducted + required):
        required += 1
    print("Current attendance:", round(attended / conducted * 100, 2), "%")
    print("Additional classes required:", required)
```

7. Test Case

Input / Condition	Expected Result
60 attended, 100 conducted	Current attendance 60.0%; 60 additional classes
Attended greater than conducted	Invalid attendance details

8. Sample Output

```
Classes attended: 60
Total classes conducted: 100
Current attendance: 60.0 %
Additional classes required: 60
```

9. Conclusion

The loop finds the smallest number of additional attended classes needed to meet the chosen target.

Case Study 3 — ATM Transaction Sequence Simulation

1. Problem Statement

Simulate a simple ATM session in which the user can deposit or withdraw money and view the updated balance.

2. Objectives

• Accept an opening balance and transaction count.

• Process deposits and withdrawals.

• Prevent invalid amounts and withdrawals exceeding the balance.

3. Assumptions

• Menu option 1 means deposit; option 2 means withdrawal.

• Amounts must be positive.

• The program is a classroom simulation and does not connect to a bank.

4. Algorithm

1. Read opening balance and number of transactions.

2. For each transaction, read the operation and amount.

3. Validate the amount and available balance.

4. Display the final balance.

5. Pseudocode

```
START
INPUT balance, number_of_transactions
REPEAT for each transaction
    INPUT choice, amount
    IF amount <= 0: PRINT invalid amount
    ELSE IF choice is deposit: ADD amount
    ELSE IF choice is withdrawal and balance is enough: SUBTRACT amount
    ELSE PRINT error
PRINT final balance
STOP
```

6. Python Source Code

```
balance = float(input("Opening balance: Rs. "))
count = int(input("Number of transactions: "))

for i in range(count):
    print("1. Deposit  2. Withdraw")
    choice = int(input("Choose transaction: "))
    amount = float(input("Amount: Rs. "))

    if amount <= 0:
        print("Invalid amount")
    elif choice == 1:
        balance += amount
        print("Deposit successful")
    elif choice == 2:
        if amount <= balance:
            balance -= amount
            print("Withdrawal successful")
        else:
            print("Insufficient balance")
    else:
        print("Invalid choice")

print("Final balance: Rs.", round(balance, 2))
```

7. Test Case

Input / Condition	Expected Result
Opening ■1000; deposit ■200; withdraw ■150	Final balance ■1050
Withdraw more than balance	Insufficient balance

8. Sample Output

```
Opening balance: Rs. 1000
Number of transactions: 2
Deposit successful
Withdrawal successful
Final balance: Rs. 1050.0
```

9. Conclusion

The program demonstrates loops, menu choices, validation, and balance updates in a basic ATM simulation.

Case Study 4 — Fuel Consumption and Route Comparison

1. Problem Statement

Compare the estimated fuel use and fuel cost of two vehicles for the same journey.

2. Objectives

• Read distance, fuel price, and mileage for two vehicles.

• Calculate fuel required and total cost.

• Identify the more economical vehicle.

3. Assumptions

• Distance, fuel price, and mileage values must be positive.

• Mileage is measured in kilometres per litre.

• The estimate excludes tolls and other vehicle costs.

4. Algorithm

1. Read journey distance and fuel price.

2. Read mileage for each vehicle.

3. Calculate litres needed and cost for each vehicle.

4. Compare the costs.

5. Pseudocode

```
START
INPUT distance, price, mileage1, mileage2
VALIDATE all values are positive
fuel1 = distance / mileage1
fuel2 = distance / mileage2
cost1 = fuel1 * price
cost2 = fuel2 * price
PRINT fuel and cost for both
COMPARE costs
STOP
```

6. Python Source Code

```
distance = float(input("Distance (km): "))
price = float(input("Fuel price per litre: Rs. "))
m1 = float(input("Vehicle 1 mileage (km/l): "))
m2 = float(input("Vehicle 2 mileage (km/l): "))

if distance <= 0 or price <= 0 or m1 <= 0 or m2 <= 0:
    print("Enter positive values")
else:
    fuel1, fuel2 = distance / m1, distance / m2
    cost1, cost2 = fuel1 * price, fuel2 * price
    print("Vehicle 1 fuel:", round(fuel1, 2), "L; cost Rs.", round(cost1, 2))
    print("Vehicle 2 fuel:", round(fuel2, 2), "L; cost Rs.", round(cost2, 2))
    if cost1 < cost2:
        print("Vehicle 1 is more economical")
    elif cost2 < cost1:
        print("Vehicle 2 is more economical")
    else:
        print("Both cost the same")
```

7. Test Case

Input / Condition	Expected Result
200 km, ■100/L, mileage 20 and 15 km/L	Vehicle 1: 10 L, ■1000; Vehicle 2: 13.33 L, ■1333.33
Zero mileage	Enter positive values

8. Sample Output

```
Distance (km): 200
Fuel price per litre: Rs. 100
Vehicle 1 fuel: 10.0 L; cost Rs. 1000.0
Vehicle 2 fuel: 13.33 L; cost Rs. 1333.33
Vehicle 1 is more economical
```

9. Conclusion

The calculations estimate fuel requirements and compare journey costs using the supplied mileage values.

Case Study 5 — Advanced Numeric Pattern Generator

1. Problem Statement

Print a symmetric numeric diamond pattern for a user-supplied height.

2. Objectives

• Accept a pattern size.

• Use nested loops to print a numeric diamond.

• Reject sizes outside the chosen safe range.

3. Assumptions

• The size must be an integer from 1 to 20.

• Each row ascends from 1 to its row number and descends back to 1.

4. Algorithm

1. Read n.

2. Validate n.

3. Print the upper half including the middle row.

4. Print the lower half in reverse order.

5. Pseudocode

```
START
INPUT n
IF n < 1 OR n > 20: PRINT invalid size
ELSE
    FOR row from 1 to n: print spaces and ascending/descending digits
    FOR row from n-1 down to 1: print spaces and digits
STOP
```

6. Python Source Code

```
n = int(input("Enter diamond size (1-20): "))

if n < 1 or n > 20:
    print("Invalid size")
else:
    for row in range(1, n + 1):
        print(" " * (n - row), end="")
        for j in range(1, row + 1):
            print(j, end="")
        for j in range(row - 1, 0, -1):
            print(j, end="")
        print()

    for row in range(n - 1, 0, -1):
        print(" " * (n - row), end="")
        for j in range(1, row + 1):
            print(j, end="")
        for j in range(row - 1, 0, -1):
            print(j, end="")
        print()
```

7. Test Case

Input / Condition	Expected Result
Size 4	Prints a 7-row symmetric diamond
Size 0	Invalid size

8. Sample Output

```
   1
  121
```

```
  12321
 1234321
  12321
   121
    1
```

9. Conclusion

The program uses nested loops and string repetition to build a symmetric numeric pattern.

Case Study 6 — Password Policy Validation Engine

1. Problem Statement

Check whether a password meets a simple policy involving length, uppercase, lowercase, digit, and special character requirements.

2. Objectives

• Read a password.

• Check each policy condition.

• Display whether the password is valid and list unmet conditions.

3. Assumptions

• Minimum length is 8 characters.

• A special character means a non-alphanumeric character.

• This classroom example does not store or transmit passwords.

4. Algorithm

1. Read the password.

2. Check length and character categories.

3. Collect missing requirements.

4. Print the result.

5. Pseudocode

```
START
INPUT password
CHECK length >= 8
CHECK uppercase, lowercase, digit, special character
IF all conditions pass: PRINT valid
ELSE PRINT invalid and missing rules
STOP
```

6. Python Source Code

```
password = input("Enter password: ")
errors = []

if len(password) < 8:
    errors.append("at least 8 characters")
if not any(c.isupper() for c in password):
    errors.append("an uppercase letter")
if not any(c.islower() for c in password):
    errors.append("a lowercase letter")
if not any(c.isdigit() for c in password):
    errors.append("a digit")
if not any(not c.isalnum() for c in password):
    errors.append("a special character")

if errors:
    print("Password is invalid. Need:", ", ".join(errors))
else:
    print("Password is valid")
```

7. Test Case

Input / Condition	Expected Result
Hello@123	Password is valid
hello123	Invalid: needs uppercase and special character

8. Sample Output

```
Enter password: Hello@123
Password is valid
```

9. Conclusion

The program checks common password rules and explains which requirements are missing.

Case Study 7 — String-Based Examination Answer Evaluation

1. Problem Statement

Evaluate a student's multiple-choice answers against a fixed answer key and report correct, incorrect, unanswered, and total score.

2. Objectives

• Read five answers.

• Compare answers with the key.

• Count correct, incorrect, and unanswered responses.

3. Assumptions

• The answer key in this sample is A, B, C, D, A.

• Each correct answer earns one point; there is no negative marking.

• A blank answer is unanswered.

4. Algorithm

1. Store the answer key.

2. Read each response and normalize to uppercase.

3. Compare the response with the key.

4. Display counts and score.

5. Pseudocode

```
START
key = [A, B, C, D, A]
FOR each question
    INPUT answer
    IF blank: unanswered += 1
    ELSE IF answer matches key: correct += 1
    ELSE: wrong += 1
PRINT counts and score
STOP
```

6. Python Source Code

```
key = ["A", "B", "C", "D", "A"]
correct = wrong = unanswered = 0

for i, expected in enumerate(key, start=1):
    answer = input(f"Answer for question {i}: ").strip().upper()
    if answer == "":
        unanswered += 1
    elif answer == expected:
        correct += 1
    else:
        wrong += 1

print("Correct:", correct)
print("Wrong:", wrong)
print("Unanswered:", unanswered)
print("Score:", correct, "/", len(key))
```

7. Test Case

Input / Condition	Expected Result
A, B, A, D, blank	Correct 3; wrong 1; unanswered 1; score 3/5
All blank	Correct 0; wrong 0; unanswered 5

8. Sample Output

```
Answer for question 1: A
Answer for question 2: B
Answer for question 3: A
Answer for question 4: D
Answer for question 5:
Correct: 3
Wrong: 1
Unanswered: 1
Score: 3 / 5
```

9. Conclusion

The program uses strings, lists, loops, and conditions to score a short multiple-choice test.

Case Study 8 — Text Compression Using Character Runs

1. Problem Statement

Compress a string by replacing consecutive repeated characters with the character followed by its run length.

2. Objectives

• Read a text string.

• Count consecutive occurrences of each character.

• Display compressed text and compare lengths.

3. Assumptions

• This is basic run-length encoding, not a general-purpose compression format.

• Input is treated as a sequence of characters including spaces.

4. Algorithm

1. Read text.

2. Start at the first character.

3. Count matching consecutive characters.

4. Append character and count to result.

5. Display original and compressed lengths.

5. Pseudocode

```
START
INPUT text
compressed = empty string
i = 0
WHILE i < length(text)
    count consecutive equal characters
    append character + count
    move i past that run
PRINT compressed and lengths
STOP
```

6. Python Source Code

```
text = input("Enter text to compress: ")
parts = []
i = 0

while i < len(text):
    count = 1
    while i + count < len(text) and text[i] == text[i + count]:
        count += 1
    parts.append(text[i] + str(count))
    i += count

compressed = "".join(parts)
print("Original text:", text)
print("Compressed text:", compressed)
print("Original length:", len(text))
print("Compressed length:", len(compressed))
```

7. Test Case

Input / Condition	Expected Result
aaabbc	Compressed text a3b2c1; original length 6; compressed length 6
abcd	Compressed text a1b1c1d1; compressed form is longer

8. Sample Output

```
Enter text to compress: aaabbc
```

```
Original text: aaabbc
Compressed text: a3b2c1
Original length: 6
Compressed length: 6
```

9. Conclusion

The program demonstrates string traversal and run counting; the encoded result may be longer than the input.

Case Study 9 — Student Marks Matrix Analysis

1. Problem Statement

Store marks for multiple students across multiple subjects and calculate each student's total and average, then identify the highest total.

2. Objectives

• Accept student and subject counts.

• Store marks in a two-dimensional list.

• Calculate totals and averages.

• Identify the student with the highest total.

3. Assumptions

• Marks are between 0 and 100.

• At least one student and one subject are entered.

• Ties for the highest total are reported as the first maximum in this sample.

4. Algorithm

1. Read dimensions.

2. Read marks into a nested list.

3. Calculate each row's sum and average.

4. Find and display the highest total.

5. Pseudocode

```
START
INPUT number_of_students, number_of_subjects
READ marks into matrix
FOR each student
    total = sum of row
    average = total / subjects
    PRINT total and average
FIND maximum total
PRINT topper
STOP
```

6. Python Source Code

```
students = int(input("Number of students: "))
subjects = int(input("Number of subjects: "))

if students <= 0 or subjects <= 0:
    print("Invalid dimensions")
else:
    marks = []
    for i in range(students):
        row = []
        for j in range(subjects):
            mark = float(input(f"Student {i+1}, subject {j+1}: "))
            row.append(mark)
        marks.append(row)

    totals = []
    for i, row in enumerate(marks, start=1):
        total = sum(row)
        average = total / subjects
        totals.append(total)
        print(f"Student {i}: total={total:g}, average={average:.2f}")

    topper = totals.index(max(totals)) + 1
    print("Topper: Student", topper)
```

7. Test Case

Input / Condition	Expected Result
2 students; marks 80,75,90 and 85,95,88	Totals 245 and 268; averages 81.67 and 89.33; Student 2 topper
0 students	Invalid dimensions

8. Sample Output

```
Student 1: total=245, average=81.67
Student 2: total=268, average=89.33
Topper: Student 2
```

9. Conclusion

The program uses nested lists and loops to analyze marks and identify the student with the highest total.

Case Study 10 — Food Waste Monitoring System

1. Problem Statement

Track food prepared and served over several days and calculate total waste and waste percentage.

2. Objectives

• Collect daily prepared and served quantities.

• Calculate daily waste.

• Summarize totals and calculate the waste percentage.

3. Assumptions

• Quantities are measured in the same unit, such as meal portions.

• Served quantity must not exceed prepared quantity.

• Waste percentage is total waste divided by total prepared, multiplied by 100.

4. Algorithm

1. Read the number of days.

2. For each day, read prepared and served quantities.

3. Validate each day's quantities.

4. Accumulate totals and calculate waste percentage.

5. Pseudocode

```
START
INPUT days
total_prepared = total_served = total_waste = 0
FOR each day
    INPUT prepared, served
    IF values invalid: report error
    ELSE update totals
waste_percent = total_waste / total_prepared * 100
PRINT totals and percentage
STOP
```

6. Python Source Code

```
days = int(input("Number of days: "))
total_prepared = total_served = total_waste = 0

if days <= 0:
    print("Invalid number of days")
else:
    for day in range(1, days + 1):
        prepared = int(input(f"Day {day} - food prepared: "))
        served = int(input(f"Day {day} - food served: "))

        if prepared < 0 or served < 0 or served > prepared:
            print("Invalid quantities; day not included")
            continue

        total_prepared += prepared
        total_served += served
        total_waste += prepared - served

    if total_prepared == 0:
        print("No valid prepared quantity to calculate percentage")
    else:
        percent = total_waste / total_prepared * 100
        print("Total prepared:", total_prepared)
        print("Total served:", total_served)
        print("Total waste:", total_waste)
        print("Waste percentage:", round(percent, 2), "%")
```

7. Test Case

Input / Condition	Expected Result
2 days: 100 prepared/85 served; 120 prepared/110 served	Prepared 220; served 195; waste 25; waste percentage 11.36%
Served greater than prepared	Invalid quantities; day not included

8. Sample Output

```
Number of days: 2
Day 1 - food prepared: 100
Day 1 - food served: 85
Day 2 - food prepared: 120
Day 2 - food served: 110
Total prepared: 220
Total served: 195
Total waste: 25
Waste percentage: 11.36 %
```

9. Conclusion

The program totals food preparation, service, and waste figures to support basic food-waste monitoring.

Appendix — Submission Checklist

• Confirm the ten selected case-study numbers and titles match the assignment PDF.

• Save each program as a separate .py file and run it in VS Code or Terminal.

• Test normal, boundary, and invalid inputs; update the test-case tables with actual results.

• Capture genuine screenshots of each program's output from your computer and insert them in the relevant sections.

• Add a flowchart for each program if required by the instructor; the pseudocode in this sample is not a graphical flowchart.

• Add the repository URL to the final report or README if requested.

• Check spelling, page numbers, student details, and all code before submitting.

Important: The output blocks in this sample are text examples, not screenshots captured from a live Python run. Use your own verified screenshots for final submission.

Suggested repository name: Python-Activity-Based-Learning